

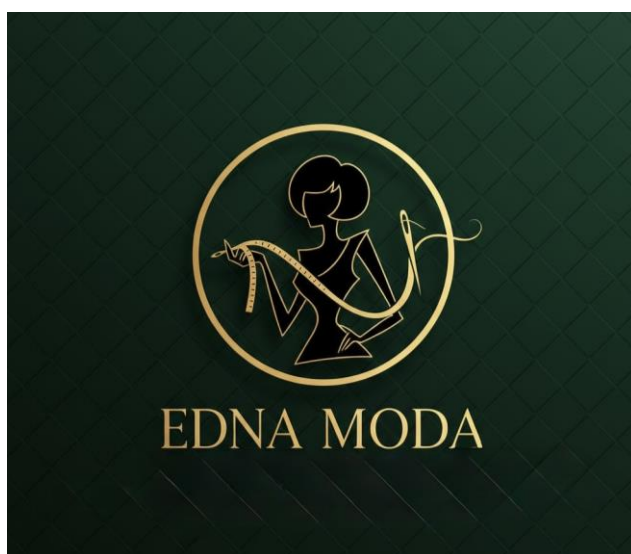


**Universidad
Europea** MADRID

PROYECTO INTEGRADOR

1DAW

EMG



Índice

Resumen.....	2
1. Introducción	3
1.1 Requisitos funcionales	4
1.2 Requisitos no funcionales	5
2. Objetivos	6
2.1 Objetivos generales	6
2.2 Objetivos específicos	7
3. Tecnologías utilizadas.....	8
3.1 Lenguaje de programación:	8
3.2 Base de datos:.....	8
3.3 Modelado y diagramación:	8
3.4 UML y análisis del sistema:.....	8
3.5 Diseño de la interfaz gráfica:	9
3.6 Control de versiones:.....	9
3.7 Gestión del proyecto:	9
4. Desarrollo e implementación	10
4.1 Diagrama E / R	10
4.2 Diagrama de la Normalización.....	11
4.3 Diagrama de casos de uso	12
4.4 Diseño de la interfaz gráfica con Figma:	13
4.5 Diseño de Logo:	14
5. Metodología	15
- Aplicación de Scrum en el proyecto.....	16
5.1 GitHub.....	16
5.2 Trello	16

5.3 Diseño del sistema	17
5.4 Diagrama de Gantt del proyecto	17
5.5 Diagrama de clase.....	19
5.6 Implementación del Modelo (MVC)	20
5.7 Implementación del Controlador	21
5.8 Conexión con la Base de Datos.....	22
6. Resultados y conclusiones.....	23
Resultados obtenidos	24
Conclusiones	24
7. Anexos	26
8.1 Anexo I – Listado de requisitos de la aplicación	26
8.1.1 RP1 – Arquitectura del sistema	26
8.1.2 RP2 – Interfaz gráfica.....	26
8.1.3 RP3 – Gestión de clientes	26
8.1.4 RP4 – Gestión de trajes.....	27
8.1.5 RP5 – Gestión de talleres.....	27
8.1.6 RP6 – Gestión de citas	27
8.1.7 RP7 – Validaciones del sistema	28
8.2 Requisitos de Base de Datos	28
8.2.1 RBD1 – Persistencia de datos	28
8.2.2 RBD2 – Tablas necesarias	28
8.2.3 RBD3 – Relaciones entre tablas.....	29
8.2.4 RBD4 – Integridad de datos	29
8.2.5 RBD5 – Datos iniciales	29
8. Anexo III: Código relevante	30

1. Clase principal – APP:Clase encargada de iniciar la aplicación y mostrar la pantalla de inicio de sesión.	30
4. Modelo de datos – Clase CitaRepresentación de la entidad Cita dentro del sistema. 32	
5. Validación de usuarios – LoginControllerComprobación de credenciales de acceso al sistema.....	32

Resumen

El objetivo principal de este proyecto es desarrollar una aplicación de escritorio que permita gestionar de forma completa y eficiente las citas del taller de Edna Moda. La finalidad es sustituir la organización manual por una herramienta informática que facilite el control y la planificación del trabajo diario del taller.

La aplicación permitirá almacenar y administrar toda la información relevante del negocio, incluyendo clientes, trajes, talleres y citas. A través de una interfaz sencilla e intuitiva, el usuario podrá crear, consultar, modificar y eliminar datos de manera rápida, mejorando la organización del tiempo y evitando errores en la gestión de la agenda.

Además, este proyecto busca aplicar de forma práctica los conocimientos adquiridos durante el curso en programación, bases de datos y entornos de desarrollo. Para ello, se utilizará el patrón de arquitectura MVC, se conectará la aplicación a una base de datos relacional y se seguirán buenas prácticas de desarrollo de software.

Otro objetivo importante es mejorar la planificación del taller mediante la automatización de procesos y la incorporación de reglas de negocio, como evitar conflictos entre clientes. En conjunto, el proyecto pretende crear una aplicación funcional, estable y útil que simule un entorno real de desarrollo profesional.

1. Introducción

Tras analizar las especificaciones del proyecto, se han identificado las entidades principales y las funcionalidades necesarias para el desarrollo de la aplicación de gestión de citas del taller de Edna Moda.

El objetivo de la aplicación es permitir a Edna organizar y gestionar las citas de sus clientes, así como controlar la información relacionada con los trajes y los talleres donde se realizan las distintas actividades del proceso de diseño y confección.

En primer lugar, se han identificado las entidades principales del sistema. Estas entidades representan los datos que serán almacenados y gestionados por la aplicación dentro de una base de datos. Las entidades principales del sistema son: Cliente, Traje, Taller y Cita.

La entidad Cliente almacena la información relacionada con los clientes del taller. Cada cliente se identifica mediante un número único y dispone de información como el nombre del superhéroe o supervillano, su superpoder y los colores característicos de su traje.

La entidad Traje representa los trajes diseñados para los clientes. Cada traje tiene un identificador único, un nombre que permite identificarlo y un estado que indica en qué fase del proceso se encuentra, como diseño, costura o en taller. Cada traje está asociado a un cliente.

La entidad Taller representa las diferentes salas del estudio de moda donde se realizan las distintas actividades. Cada taller tiene un identificador único, un nombre inspirado en ciudades importantes de la moda y un tipo de sala que indica su función, como diseño, costura o pruebas.

Finalmente, la entidad Cita representa las reservas realizadas por los clientes para acudir al taller. Cada cita contiene información sobre el cliente, el traje asociado, el taller donde se realizará la actividad y la fecha, hora y duración de la cita.

En cuanto a la funcionalidad del sistema, la aplicación permitirá realizar operaciones básicas de gestión de datos (CRUD), como crear, consultar, modificar y eliminar clientes, trajes, talleres y citas.

A partir del análisis realizado, se han definido los requisitos funcionales y no funcionales que describen el comportamiento y las características que debe cumplir el sistema de gestión de citas del taller de Edna Moda.

1.1 Requisitos funcionales

Los requisitos funcionales describen las funcionalidades principales que debe ofrecer la aplicación para permitir la gestión de clientes, trajes, talleres y citas dentro del sistema.

El sistema deberá permitir gestionar la información de los clientes del taller. Esto incluye la posibilidad de crear nuevos clientes, consultar la información de los clientes registrados, modificar sus datos y eliminar clientes del sistema cuando sea necesario.

Además, el sistema deberá permitir gestionar los trajes asociados a los clientes. Para cada traje será posible crear nuevos registros, consultar su información, modificar su estado dentro del proceso de confección y eliminarlo si es necesario.

El sistema también deberá permitir gestionar los talleres donde se realizan las distintas actividades del proceso de diseño y confección. Los usuarios podrán crear nuevos talleres, consultar los talleres existentes, modificar su información y eliminar talleres del sistema.

Otra funcionalidad principal del sistema es la gestión de citas. El sistema deberá permitir crear citas para los clientes del taller. Cada cita deberá contener información sobre el cliente, el traje asociado, el taller donde se realizará la actividad, así como la fecha, la hora y la duración de la cita.

Asimismo, el sistema deberá permitir modificar las citas existentes. Esto incluye la posibilidad de cambiar la fecha y la hora de la cita, el taller asignado, el traje relacionado o el cliente asociado. Finalmente, el sistema deberá permitir eliminar citas cuando sea necesario.

1.2 Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad y las restricciones que debe cumplir el sistema.

El sistema deberá proporcionar una interfaz clara, intuitiva y fácil de utilizar que permita a los usuarios gestionar la información del taller de manera eficiente.

Asimismo, el sistema deberá funcionar de manera estable durante su uso normal, evitando fallos inesperados o interrupciones en su funcionamiento.

Todos los datos de la aplicación, incluyendo clientes, trajes, talleres y citas, deberán almacenarse en una base de datos relacional y mantenerse disponibles entre diferentes sesiones del sistema.

El sistema también deberá garantizar la protección de los datos almacenados, evitando accesos no autorizados a la información.

En términos de rendimiento, la aplicación deberá responder a las acciones del usuario en un tiempo razonable durante su uso normal.

Además, todas las pantallas de la aplicación deberán seguir un diseño coherente para proporcionar una experiencia de usuario uniforme.

Por último, el sistema deberá desarrollarse con una arquitectura organizada que facilite su mantenimiento y permita futuras mejoras o ampliaciones.

2. Objetivos

2.1 Objetivos generales

El objetivo principal de este proyecto es diseñar y desarrollar una aplicación de escritorio que permita gestionar de forma integral las citas del taller de Edna Moda, automatizando los procesos de organización del trabajo y sustituyendo la gestión manual por un sistema informático fiable.

La aplicación debe facilitar la administración de toda la información relacionada con el negocio, incluyendo clientes, trajes, talleres y citas, permitiendo realizar operaciones de alta, consulta, modificación y eliminación de datos de forma rápida y sencilla.

Además, se pretende aplicar los conocimientos adquiridos durante el curso en las áreas de programación, bases de datos y entornos de desarrollo, utilizando una arquitectura basada en el patrón MVC, conexión con bases de datos relacionales y buenas prácticas de desarrollo de software.

Con este proyecto se busca crear una herramienta funcional, usable y organizada que mejore la planificación del trabajo del taller, reduzca errores humanos en la gestión de citas y sirva como experiencia práctica de desarrollo de una aplicación completa.

2.2 Objetivos específicos

- **Gestionar clientes**
 - a. Añadir nuevos clientes.
 - b. Modificar sus datos.
 - c. Eliminar clientes.
 - d. Consultar el listado de clientes.
- **Gestionar trajes**
 - a. Crear trajes asociados a cada cliente.
 - b. Modificar el estado del traje (diseño, costura, taller).
 - c. Eliminar trajes.
 - d. Consultar los trajes existentes.
- **Gestionar talleres**
 - a. Añadir salas del estudio.
 - b. Editar sus datos.
 - c. Eliminar talleres.
 - d. Consultar los talleres disponibles.
- **Gestionar citas**
 - a. Reservar nuevas citas.
 - b. Modificar citas existentes.
 - c. Cancelar citas.
 - d. Visualizar la agenda de citas.
- **Controlar el acceso a la aplicación**
 - a. Implementar una pantalla de login para el acceso exclusivo de Edna.
- **Evitar conflictos entre clientes (funcionalidad bonus)**
 - a. Diferenciar entre héroes y villanos.
 - b. Impedir que tengan citas consecutivas sin una hora de margen.
- **Conectar la aplicación con una base de datos**
 - a. Guardar la información de forma permanente.
- Permitir consultas y actualizaciones de datos.

3. Tecnologías utilizadas

Para el desarrollo del proyecto se han utilizado las siguientes herramientas y tecnologías:

3.1 Lenguaje de programación:

- **Java (JDK actualizado)**
Lenguaje principal para el desarrollo de la aplicación orientada objetos.
- **Eclipse IDE**
Utilizado para programar, compilar y depurar la aplicación.

3.2 Base de datos:

- **MySQL**
Sistema gestor de bases de datos relacional donde se almacenará toda la información de la aplicación.

3.3 Modelado y diagramación:

- **Drow.io**
Herramienta utilizada para la elaboración de:
 - Diagrama Entidad-Relación.
 - Diagrama de normalización.

3.4 UML y análisis del sistema:

- **Visual Paradigma Online**
Herramienta utilizada para la creación de los diagramas UML del proyecto:
 - Diagrama de casos de uso.

3.5 Diseño de la interfaz gráfica:

- **Figma.**

Herramienta de diseño de interfaces (UI /UX) que permite crear prototipos visuales de aplicaciones antes de desarrollarlas.

3.6 Control de versiones:

- **GitHub**

Repositorio del proyecto para el control de versiones y seguimiento del desarrollo.

3.7 Gestión del proyecto:

- **Trello**

Herramienta usada para organizar tareas y planificación del sprint mediante metodologías SCRUM.

4. Desarrollo e implementación

4.1 Diagrama E / R

El diagrama E/R respalda la estructura conceptual de la base de datos utilizando en la aplicación. En él se muestran las entidades principales de sistema, sus atributos y las relaciones que existen entre ellas.

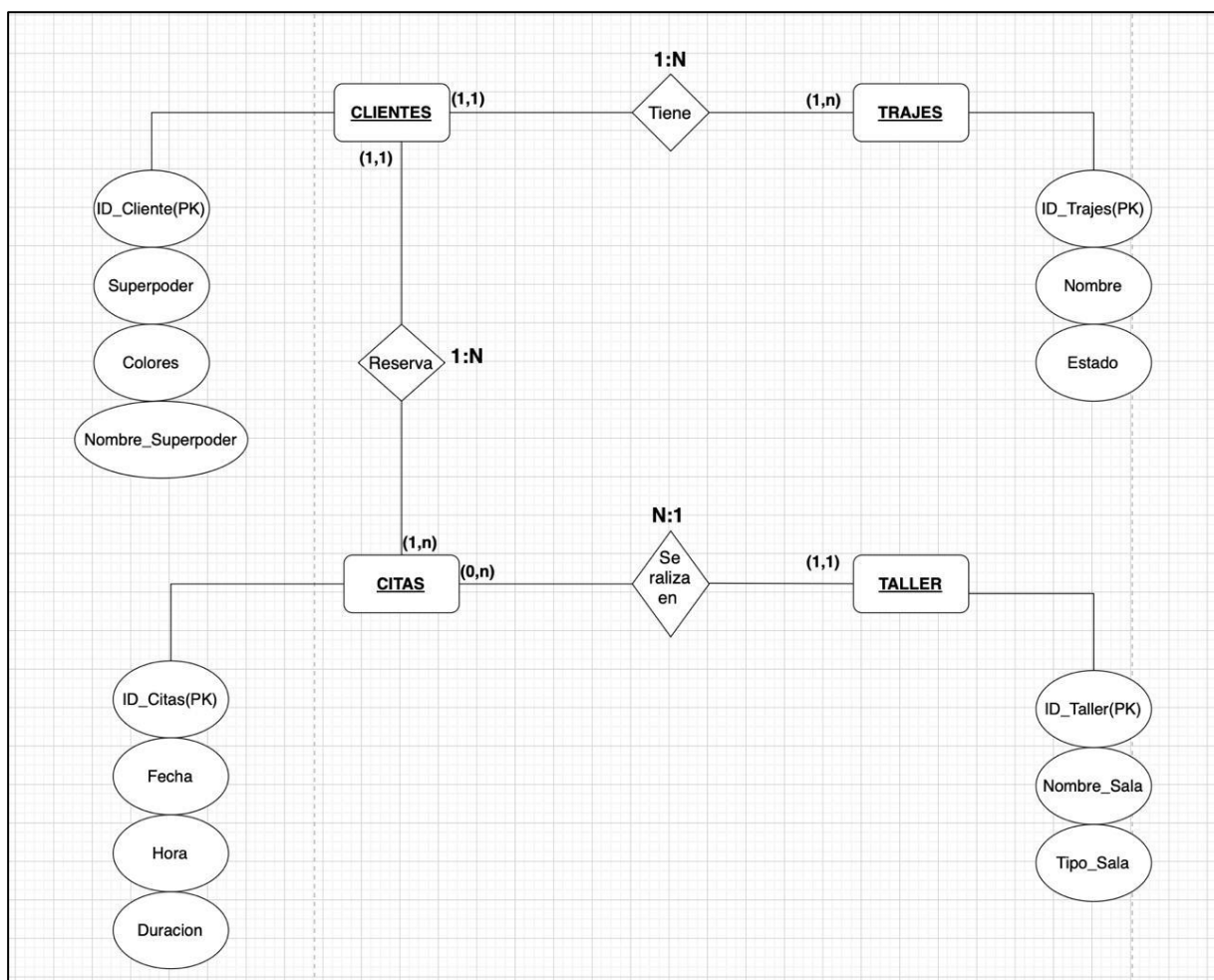


Figura 1. Diagrama E/R del sistema de gestión de citas.

4.2 Diagrama de la Normalización

- La base de datos ha sido normalizada hasta la Tercera Forma Normal (3NF).
- En primer lugar, se cumple la Primera Forma Normal (1NF) porque cada tabla tiene una clave primaria y todos los atributos contienen valores atómicos, sin grupos repetidos.
- En segundo lugar, se cumple la Segunda Forma Normal (2NF) porque todos los atributos no clave dependen completamente de la clave primaria, y no existen dependencias parciales.
- Por último, se cumple la Tercera Forma Normal (3NF) porque no hay dependencias transitivas. Todos los atributos no clave dependen únicamente de la clave primaria de su respectiva tabla.
- Esta normalización reduce la redundancia de datos y mejora la integridad de la información.

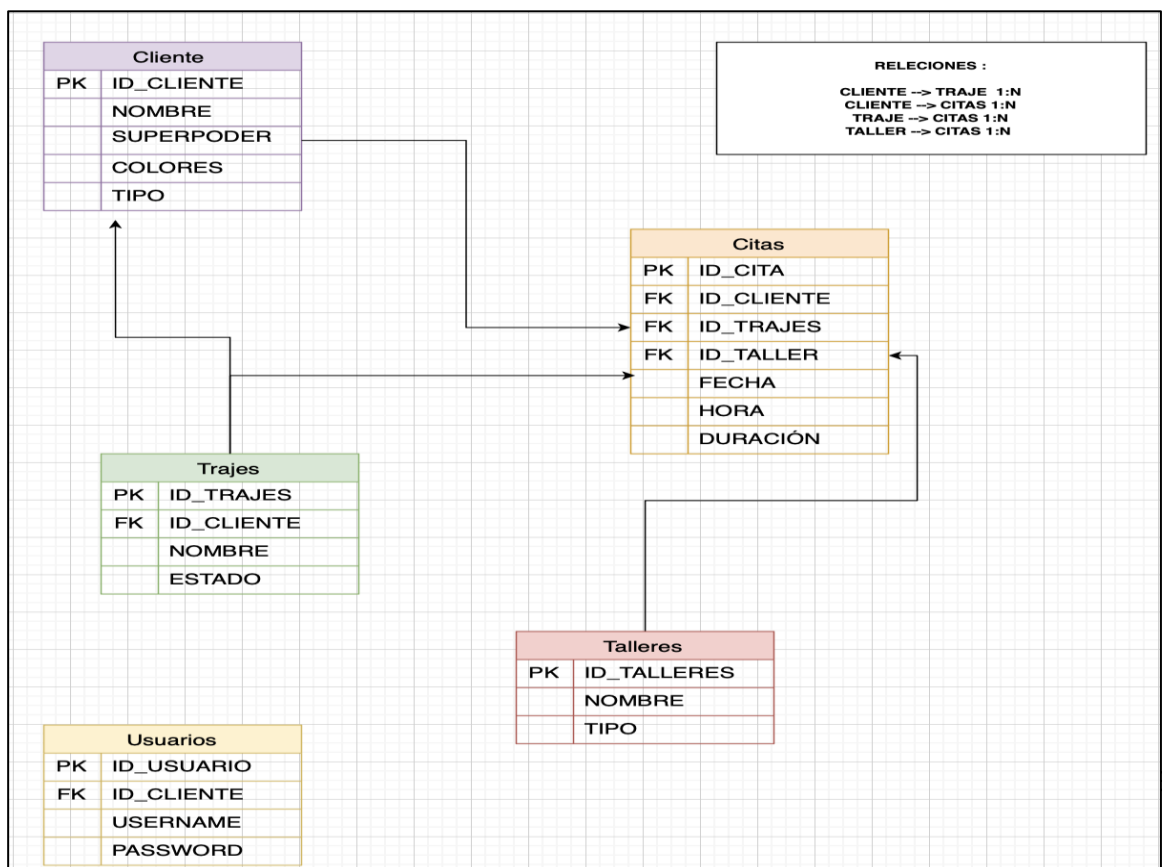


Figura 2. Diagrama Normalización del sistema de gestión de citas.

4.3 Diagrama de casos de uso

Este diagrama representa las funcionalidades principales de la aplicación de gestión de citas de taller de Edna Moda y cómo interactúa la usuaria con el sistema.

Durante este sprint se ha elaborado el diagrama de casos de uso, que representa las funcionalidades del sistema desde el punto de vista del usuario.

Este diagrama muestra:

- La interacción de Edna Moda con la aplicación.
- Las principales funcionalidades del sistema:
 - Gestión de clientes
 - Gestión de citas
 - Gestión de talleres
 - Validación de citas (regla héroe-villano)

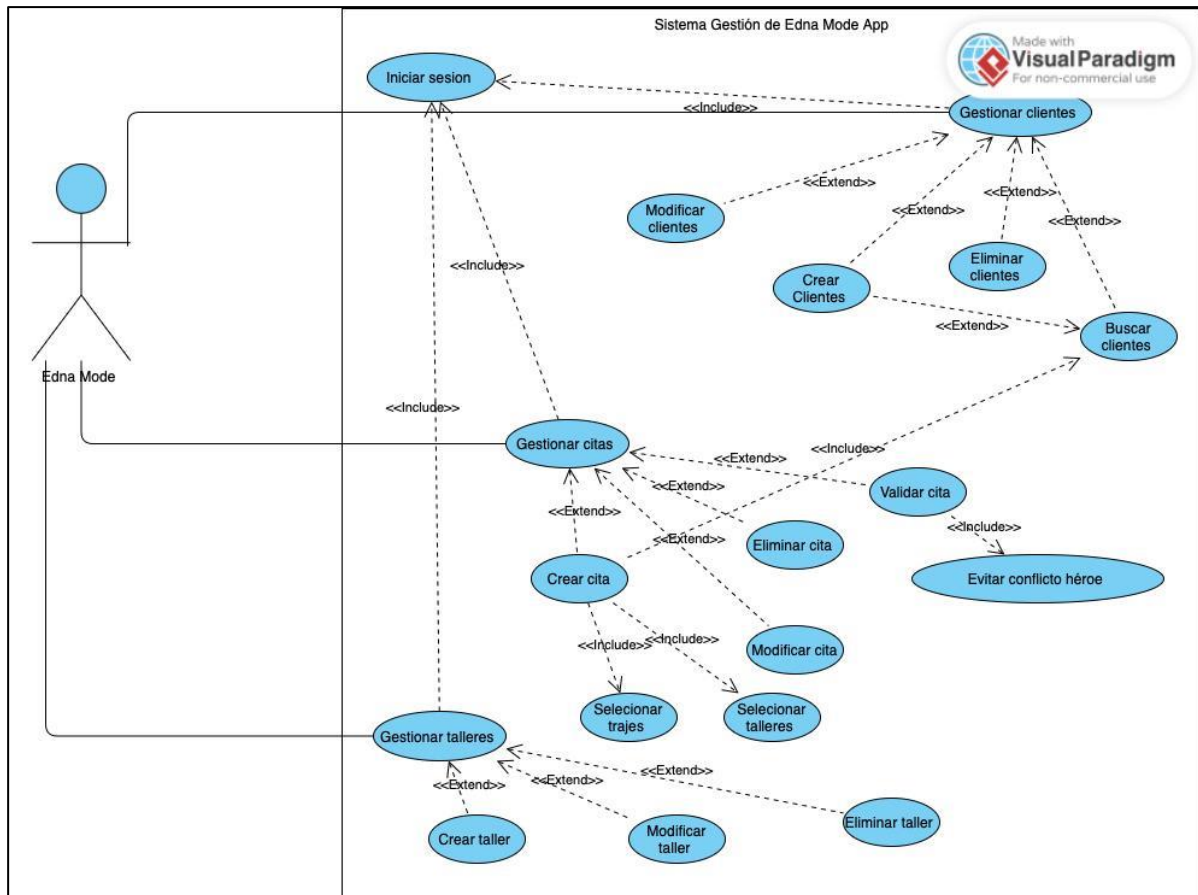


Figura 3. Diagrama de casos de uso.

4.4 Diseño de la interfaz gráfica con Figma:

Se trata de una herramienta muy utilizada en el desarrollo moderno de software porque permite diseñar la apariencia y la experiencia del usuario de forma visual.

Durante el Sprint 2 se ha utilizado Figma para diseñar la interfaz gráfica de la aplicación de gestión de citas del taller de Edna Moda.

El objetivo de este diseño previo es:

- Definir la estructura de las ventanas.
- Planificar la navegación entre pantallas.
- Mejorar la usabilidad de la aplicación.
- Reducir errores antes de la fase de programación.

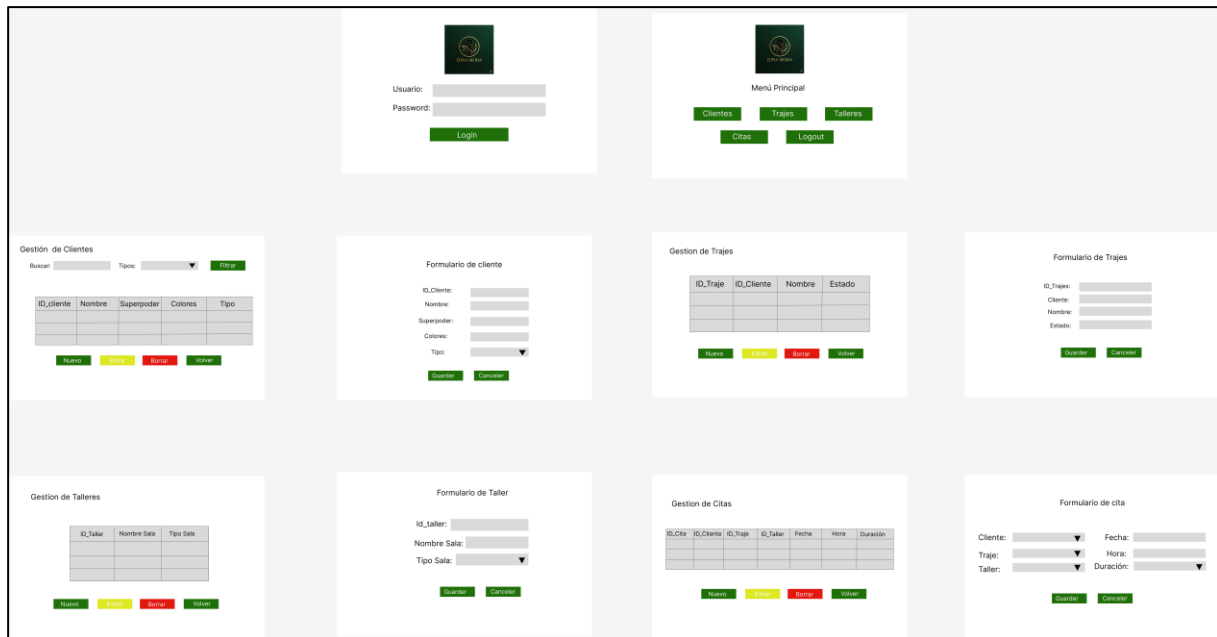


Figura 4. Diseño de la interfaz gráfica con Figma.

4.5 Diseño de Logo:

El uso de inteligencia artificial para el diseño gráfico ofrece varias ventajas:

- Generación rápida de propuestas visuales.
- Posibilidad de iterar diferentes estilos.
- Ahorro de tiempo en la fase de diseño.
- Obtención de resultados creativos sin necesidad de herramientas avanzadas de diseño.

El logo generado ha sido posteriormente seleccionado y adaptado para su uso en la aplicación.

Se ha elegido este logotipo porque refleja claramente la temática de la aplicación y ayuda a crear una identidad visual coherente. Además, transmite profesionalidad y hace que la aplicación resulte más atractiva visualmente para el usuario.

transmite. El verde está asociado con la creatividad, la innovación y el crecimiento, valores que encajan con la idea de diseño y confección de trajes personalizados.

Además, este color transmite equilibrio y profesionalidad, lo que ayuda a reforzar la identidad visual de la marca y a generar confianza en los usuarios de la aplicación.

El uso del verde también permite diferenciar la aplicación visualmente y aporta una imagen moderna y elegante acorde con el mundo de la moda y el diseño.



Figura 5. Diseño de logo.

5. Metodología

Para el desarrollo de la aplicación se ha utilizado la metodología ágil **Scrum**, una forma de trabajo iterativa e incremental que permite organizar el proyecto en pequeñas fases llamadas *sprints*.

Scrum se basa en la planificación progresiva del trabajo, la colaboración entre los miembros del equipo y la entrega continua de resultados funcionales. Esta metodología permite adaptarse a cambios, mejorar la organización del tiempo y realizar un seguimiento constante del progreso del proyecto.

- Aplicación de Scrum en el proyecto

El desarrollo del sistema se ha dividido en varios **sprints**, cada uno con una duración determinada y con objetivos concretos.

En cada sprint se han seguido las siguientes fases:

- **Planificación del sprint:** selección de tareas a realizar.
- **Desarrollo:** realización del trabajo planificado.
- **Revisión:** comprobación de los resultados obtenidos.
- **Retrospectiva:** valoración del trabajo realizado y posibles mejoras.

5.1 GitHub

El código fuente del proyecto y el progreso de desarrollo se gestionan mediante GitHub. El repositorio se utiliza para el control de versiones, lo que permite registrar y seguir los cambios realizados durante el proceso de desarrollo. En él se almacena la estructura del proyecto, el código fuente y la documentación, permitiendo actualizaciones organizadas mediante commits. Además, el repositorio de GitHub sirve como un lugar central para acceder a la última versión del proyecto y revisar su historial de desarrollo, el link de Repositorio del proyecto:

<https://github.com/sondoshany097-hub/EdnaModeAppProyecto.git>

5.2 Trello

La aplicación Trello se utilizó para la gestión y organización del proyecto. A través de tableros, listas y tarjetas, se planificaron las tareas, se siguió el progreso del desarrollo y se estructuraron las diferentes etapas del proyecto. Esta herramienta permitió mantener una visión clara de las actividades pendientes, en progreso y completadas durante todo el proceso de desarrollo, el Link de Tablero de trabajo (Trello):

<https://trello.com/b/OZA5g2uX/edana-model-a>

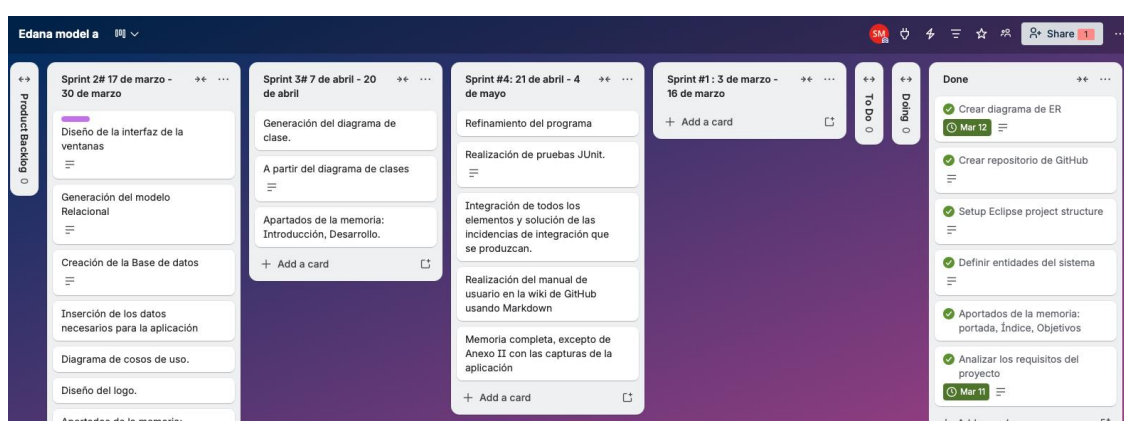


Figura 6. Trello.

5.3 Diseño del sistema

Durante el segundo sprint se ha trabajado en el diseño del sistema antes de comenzar la programación:

- Diseño de la interfaz gráfica con Figma.
- Elaboración del diagrama de casos de uso (UML).
- Diseño del modelo relacional y normalización.
- Creación de la base de datos en MySQL.
- Inserción de datos iniciales para pruebas.

5.4 Diagrama de Gantt del proyecto

Para la planificación temporal del proyecto se ha elaborado un **diagrama de Gantt**, que permite representar de forma visual la organización del trabajo a lo largo del tiempo y la distribución de las tareas en los distintos sprints.

El diagrama muestra la duración de cada fase del proyecto, el orden en el que se han realizado las actividades y la relación entre ellas. Gracias a esta herramienta ha sido posible tener una visión global del progreso del proyecto y comprobar si se estaban cumpliendo los plazos establecidos.

En el Gantt se han incluido las siguientes fases principales:

- **Planificación inicial del proyecto**
Definición de la idea, análisis de requisitos y organización del trabajo.
- **Sprint 1 – Análisis y diseño**
Elaboración de requisitos, diseño de la base de datos, normalización y primeros diagramas UML.
- **Sprint 2 – Diseño y prototipado**
Creación del diagrama de clases, diseño de la interfaz en Figma y definición del flujo de la aplicación.

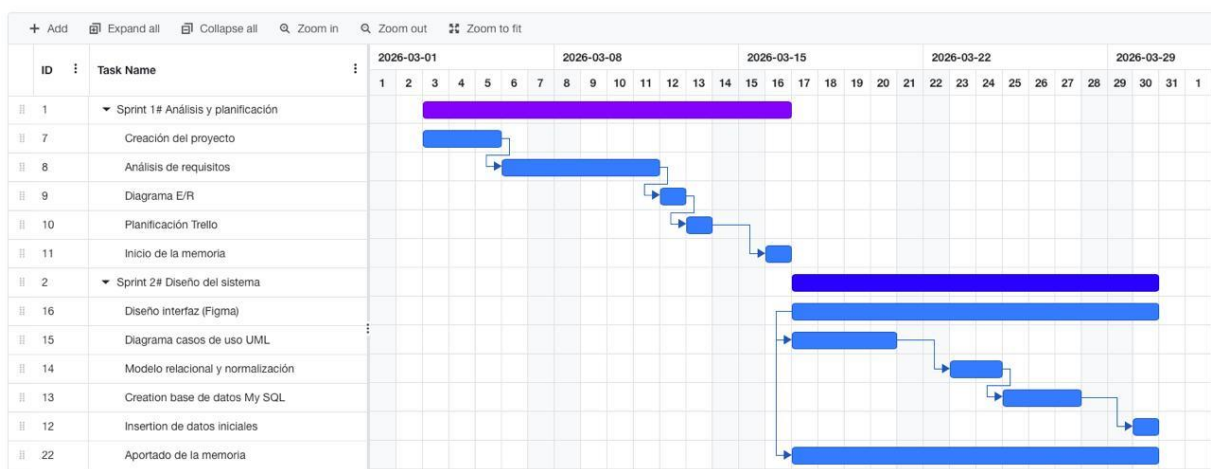


Figura 7. Gantt

- **Sprint 3 – Desarrollo**
Implementación de la base de datos en MySQL e inicio del desarrollo de la aplicación.
- **Sprint 4 – Pruebas y documentación**
Validación del sistema, inserción de datos iniciales y elaboración de la memoria final.

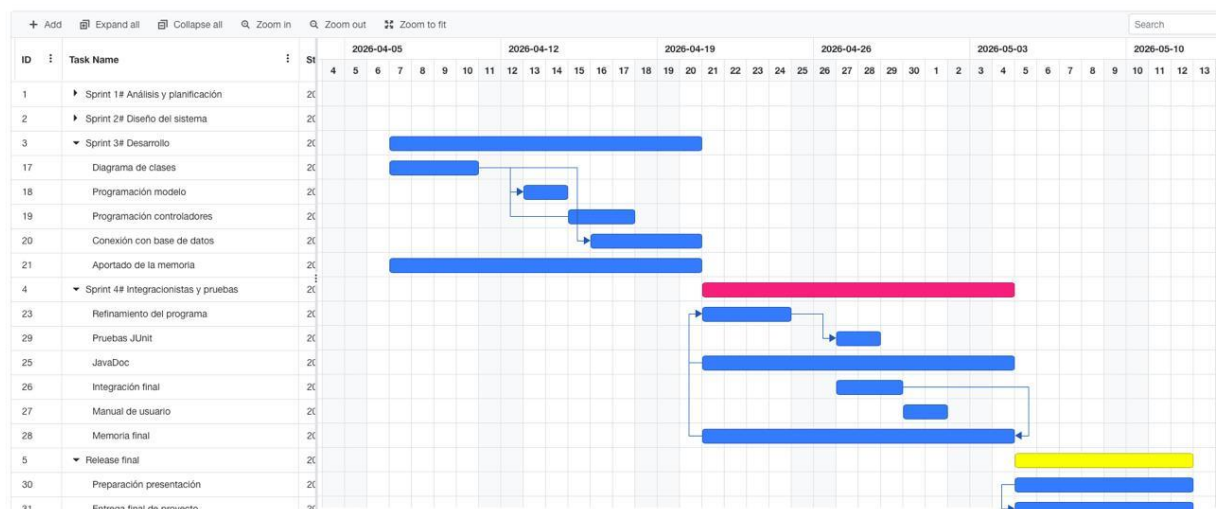


Figura 8. Gantt

El uso del diagrama de Gantt ha permitido organizar el tiempo de forma realista, repartir el trabajo entre los miembros del equipo y asegurar que cada sprint tuviera unos objetivos claros y alcanzables.

5.5 Diagrama de clase

El diagrama de clases representa la estructura del sistema y las relaciones entre las entidades principales de la aplicación. Este modelo ha servido como base para la creación de la base de datos y la posterior implementación del sistema.

El diagrama se ha realizado utilizando Visual Paradigma Online.

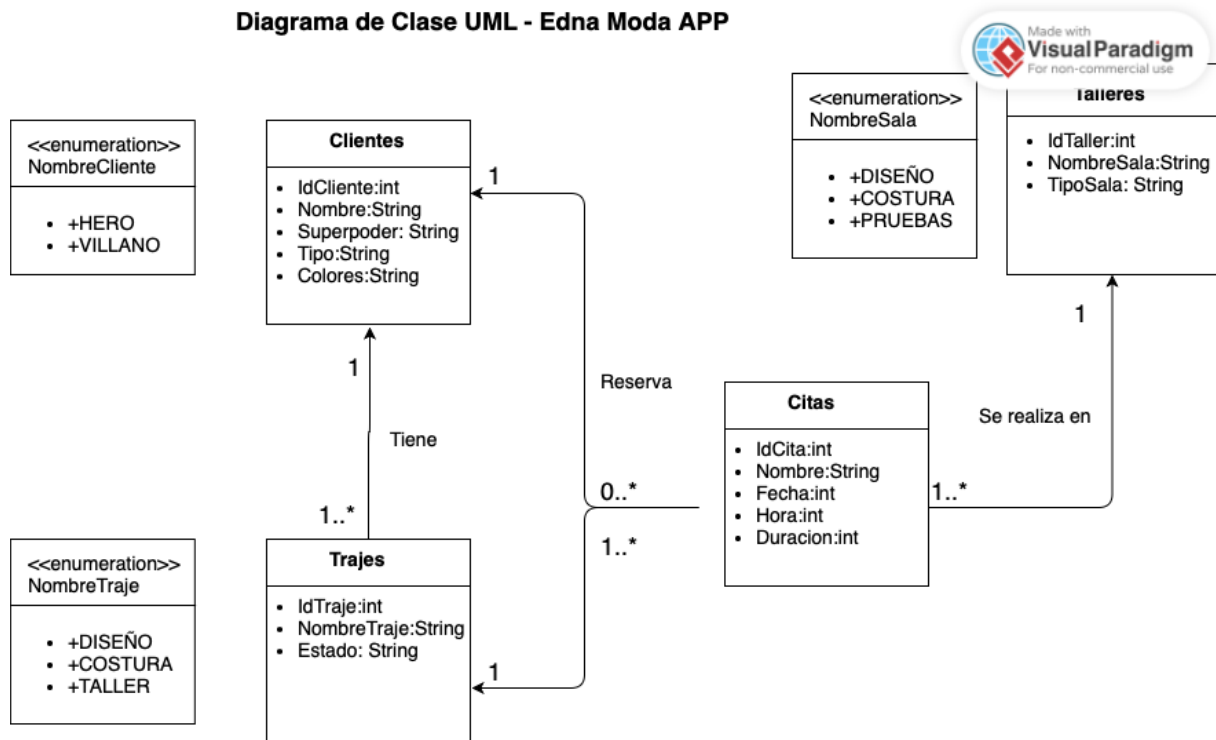


Figura 9. Diagrama de Clase

5.6 Implementación del Modelo (MVC)

Una vez definido el diagrama de clases, se han desarrollado las **clases del Modelo**, encargadas de representar los datos de la aplicación.

Cada entidad de la base de datos se corresponde con una clase Java:

- **Clase Cliente** : Gestiona la información de los clientes del taller:
 - idCliente

- nombreHeroeVillano
- superpoder
- colores
- tipo (héroe o villano – funcionalidad bonus)

- **Clase Traje:** Representa los trajes asociados a cada cliente:
 - idTraje
 - nombre
 - estado (diseño, costura, taller)
 - relación con Cliente

- **Clase Taller:** *Gestiona las salas del estudio:*
 - idTaller
 - nombreSala
 - tipoSala (diseño, costura, pruebas)

- **Clase Cita:** Elemento central de la aplicación:
 - idCita
 - cliente
 - traje
 - taller
 - fecha
 - hora
 - duración

Estas clases permiten trabajar con los datos de forma orientada a objetos y preparan la aplicación para la conexión con la base de datos.

5.7 Implementación del Controlador

Tras completar el modelo, se han desarrollado las clases del **Controlador**, encargadas de la lógica de negocio y de la comunicación con la base de datos.

Se han implementado controladores para:

- ClienteController
- TrajeController
- TallerController
- CitaController
- LoginController

Las responsabilidades principales del controlador son:

- Ejecutar operaciones CRUD:
 - Crear registros
 - Consultar registros
 - Modificar registros
 - Eliminar registros
- Validar datos antes de enviarlos a la base de datos.
- Coordinar la comunicación entre la Vista y el Modelo.

5.8 Conexión con la Base de Datos

Durante este sprint se ha configurado la conexión entre la aplicación Java y la base de datos MySQL mediante JDBC.

Se ha creado una clase de conexión encargada de:

- Establecer la conexión con la BBDD.
- Gestionar la apertura y cierre de conexiones.
- Centralizar el acceso a los datos.

Gracias a esta conexión, el sistema ya es capaz de comenzar a trabajar con datos reales almacenados en la base de datos.

6. Resultados y conclusiones

Resultados obtenidos

En el desarrollo de este proyecto se han alcanzado los objetivos planteados inicialmente, obteniendo resultados satisfactorios en relación con el trabajo realizado. A lo largo del proceso, se llevó a cabo la recopilación, análisis y tratamiento de la información necesaria para dar respuesta al problema o tema propuesto.

En primer lugar, se logró [describir el objetivo principal del proyecto: por ejemplo, investigar, analizar, diseñar o estudiar el tema], lo que permitió comprender de manera más profunda el contenido trabajado. Durante la elaboración del proyecto, se obtuvieron los siguientes resultados relevantes:

- Se identificaron los aspectos más importantes relacionados con el tema.
- Se aplicaron los métodos o herramientas previstos en la planificación inicial.
- La información recopilada permitió desarrollar un análisis coherente y estructurado.
- Se encontraron algunas limitaciones, como [falta de tiempo, dificultad para encontrar información, complejidad del tema, etc.], que influyeron en el desarrollo del trabajo.

En general, los resultados obtenidos permiten afirmar que el trabajo realizado ha sido útil para comprender mejor el tema tratado y responder a los objetivos propuestos.

Conclusiones

Tras finalizar el proyecto, se concluye que el trabajo individual ha sido una experiencia positiva a nivel académico y personal. Se han cumplido los objetivos establecidos inicialmente y se ha conseguido profundizar en el tema seleccionado.

Además, la realización del proyecto ha permitido desarrollar habilidades importantes como la organización del trabajo, la búsqueda de información, el análisis crítico y la gestión del tiempo.

Por otro lado, se han identificado algunos aspectos que podrían mejorarse en futuros trabajos. Entre ellos destacan:

- Una mayor planificación desde el inicio para evitar acumulación de trabajo al final.
- Ampliar la búsqueda de fuentes para obtener información más completa.
- Profundizar en ciertos apartados que, por limitaciones de tiempo, no pudieron desarrollarse en detalle.

En conclusión, este proyecto ha permitido alcanzar los objetivos propuestos y ha contribuido al aprendizaje del tema estudiado. También ha sido una experiencia útil para mejorar competencias necesarias en trabajos futuros.

7. Anexos

8.1 Anexo I – Listado de requisitos de la aplicación

En este anexo se recogen los requisitos necesarios desde el punto de vista de **programación y bases de datos** para garantizar el correcto funcionamiento de la aplicación de gestión de citas del taller de Edna Moda.

8.1.1 RP1 – Arquitectura del sistema

La aplicación deberá desarrollarse siguiendo el patrón **MVC (Modelo-Vista-Controlador)**, separando:

- Vista → Interfaz gráfica
- Modelo → Acceso y representación de datos
- Controlador → Lógica de la aplicación

8.1.2 RP2 – Interfaz gráfica

La aplicación deberá disponer de una interfaz gráfica de escritorio que permita:

- Pantalla de login.
- Navegación entre ventanas.
- Formularios para operaciones CRUD.
- Visualización de listados de datos.

8.1.3 RP3 – Gestión de clientes

El sistema deberá permitir:

- Alta de clientes
- Modificación de clientes
- Eliminación de clientes
- Búsqueda y listado de clientes

Cada cliente deberá contener:

- ID único
- Nombre
- Superpoder
- Colores del traje
- Tipo (héroe o villano).

8.1.4 RP4 – Gestión de trajes

El sistema deberá permitir:

- Crear trajes asociados a un cliente.
- Modificar trajes.
- Eliminar trajes.
- Consultar trajes de un cliente.

Estados posibles del traje:

- Diseño
- Costura
- Taller

8.1.5 RP5 – Gestión de talleres

El sistema deberá permitir gestionar los talleres disponibles:

- Crear taller
- Modificar taller
- Eliminar taller
- Listar talleres

Cada taller tendrá:

- ID único
- Nombre de sala
- Tipo de sala (diseño, costura, pruebas)

8.1.6 RP6 – Gestión de citas

El sistema deberá permitir:

- Crear citas
- Modificar citas
- Eliminar citas
- Consultar agenda

Datos de la cita:

- Cliente
- Traje
- Taller
- Fecha
- Hora
- Duración (1h por defecto)

8.1.7 RP7 – Validaciones del sistema

El sistema deberá validar:

- Campos obligatorios.
- Formatos de fecha y hora.
- Evitar solapamiento de citas.
- Regla héroe-villano:
 - Debe existir 1 hora de margen entre citas de distinto tipo.

8.2 Requisitos de Base de Datos

8.2.1 RBD1 – Persistencia de datos

Toda la información deberá almacenarse en una base de datos relacional MySQL.

8.2.2 RBD2 – Tablas necesarias

La base de datos deberá contener al menos las siguientes tablas:

- Clientes
- Trajes
- Talleres
- Citas

8.2.3 RBD3 – Relaciones entre tablas

El modelo deberá incluir relaciones:

- Cliente → Trajes (1:N)
- Cliente → Citas (1:N)
- Traje → Citas (1:N)
- Taller → Citas (1:N)

8.2.4 RBD4 – Integridad de datos

La base de datos deberá garantizar:

- Claves primarias.
- Claves foráneas.
- Restricciones de integridad referencial.
- Índices para optimizar consultas.

8.2.5 RBD5 – Datos iniciales

Se han insertado datos de prueba directamente en **MySQL** mediante sentencias SQL.

8. Anexo III: Código relevante

En este anexo se presentan algunos fragmentos de código representativos de la aplicación **Edna** **Moda**. No se incluye todo el código del proyecto, únicamente las partes más importantes para comprender su funcionamiento interno.

1. **Clase principal – APP:**
Clase encargada de iniciar la aplicación y mostrar la pantalla de inicio de sesión.

```
package App;
import vista.LoginView;

/**
 * Clase principal de la aplicación
 * Se encarga de iniciar la interfaz gráfica
 */
public class Main {
    public static void main(String[] args) {
        new LoginView().setVisible(true);
    }
}
```

2. **Conexión a la base de datos – ConexionDAO**
Clase responsable de establecer la conexión con la base de datos MySQL mediante JDBC.

```
package dao;
import java.sql.*;

/**
 * Clase encargada de gestionar la conexión a la base de datos MySQL.
 */
public class ConexionBD {
    private String driver = "com.mysql.cj.jdbc.Driver";

    /**
     * URL de conexión a la base de datos
     */
    private static final String URL = "jdbc:mysql://localhost/Proyecto_EdnaMode";

    /**
     * Usuario de la base de datos
     */
    private static final String USER = "root";

    /**
     * Contraseña de la base de datos
     */
    private static final String PASSWORD = "Root1234";

    /**
     * Establece y devuelve una conexión con la base de datos.
     * @return Connection si la conexión es exitosa, null si falla.
     */
}
```

3. Acceso a datos – ClienteDAO (Operaciones CRUD)

Ejemplo de operaciones básicas sobre la tabla de clientes.

```
package dao;

import Modelo.*;

/* Clase DAO encargada de gestionar las operaciones CRUD */
public class ClienteDAO {

    /* Inserta un nuevo cliente en la base de datos. */
    public int guardarCliente(Cliente cliente) {
        String sql = "INSERT INTO CLIENTES (NOMBRE, SUPERPODER, COLORES, TIPO) VALUES (?, ?, ?, ?)";

        try (Connection con = ConexionBD.conectar();
            PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

            ps.setString(1, cliente.getNombreHero());
            ps.setString(2, cliente.getSuperpoder());
            ps.setString(3, cliente.getColores());
            ps.setString(4, cliente.getTipoPersonaje());

            int filas = ps.executeUpdate();

            if (filas > 0) {
                ResultSet rs = ps.getGeneratedKeys();
                if (rs.next()) {
                    return rs.getInt(1);
                }
            }

        } catch (SQLException e) {
            System.out.println("Error en guardar cliente: " + e.getMessage());
        }

        return -1;
    }

    /* Inserta un nuevo cliente en la base de datos. */
    public List<Cliente> listarClientes() {
        List<Cliente> lista = new ArrayList<>();
        String sql = "SELECT * FROM CLIENTES";

        try (Connection con = ConexionBD.conectar();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                Cliente cliente = new Cliente();
                cliente.setIdCliente(rs.getInt("ID_CLIENTE"));
                cliente.setNombreHero(rs.getString("NOMBRE"));
                cliente.setSuperpoder(rs.getString("SUPERPODER"));
                cliente.setColores(rs.getString("COLORES"));
                cliente.setTipoPersonaje(rs.getString("TIPO"));

                lista.add(cliente);
            }

        } catch (SQLException e) {
            System.out.println("Error lista de clientes: " + e.getMessage());
        }

        return lista;
    }
}
```

```
/* Obtiene todos los clientes registrados en la base de datos. */
public boolean actualizarCliente(Cliente cliente) {
    String sql = "UPDATE CLIENTES SET NOMBRE = ?, SUPERPODER = ?, COLORES = ?, TIPO = ? WHERE ID_CLIENTE = ?";

    try (Connection con = ConexionBD.conectar();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, cliente.getNombreHero());
        ps.setString(2, cliente.getSuperpoder());
        ps.setString(3, cliente.getColores());
        ps.setString(4, cliente.getTipoPersonaje());
        ps.setInt(5, cliente.getIdCliente());

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.out.println("Error actualizar cliente: " + e.getMessage());
        return false;
    }
}

/* Elimina un cliente por su ID. */
public boolean eliminarCliente(int idCliente) {
}
```

4. Modelo de datos – Clase Cita

Representación de la entidad Cita dentro del sistema.

```
package Modelo;
/**
 * Clase que representa una cita.
 * Define los atributos y métodos (getters/setters) necesarios.
 */
public class Cita {
    private int IdCita;
    private int IdCliente;
    private int IdTraje;
    private int IdTaller;
    private String Fecha;
    private String Hora;
    private int Duracion;

    /**
     * Constructor vacío
     */
    public Cita() {
    }

    /**
     * Constructor con parámetros
     * @param IdCita
     * @param IdCliente
     * @param IdTraje
     * @param IdTaller
     * @param Fecha
     * @param Hora
     * @param Duracion
     */
    public Cita(int IdCita, int IdCliente, int IdTraje, int IdTaller, String Fecha, String Hora, int Duracion) {
        this.IdCita = IdCita;
        this.IdCliente = IdCliente;
        this.IdTraje = IdTraje;
        this.IdTaller = IdTaller;
        this.Fecha = Fecha;
        this.Hora = Hora;
        this.Duracion = Duracion;
    }
}
```

5. Validación de usuarios – LoginController

Comprobación de credenciales de acceso al sistema.

```
package Controlador;
import dao.*;

/**
 * Clase controlador encargada de gestionar el inicio de sesión.
 */
public class LoginController {
    /**
     * Objeto DAO para acceder a los datos de usuarios
     */
    private UsuarioDAO usuarioDAO;

    /**
     * Constructor: inicializa el objeto UsuarioDAO
     */
    public LoginController() {
        this.usuarioDAO = new UsuarioDAO();
    }
}
```